UNIVERSIDAD NACIONAL DE LOJA · COMPUTACIÓN · 2.º CICLO

Diseño de Circuitos

Unidad 1 · Análisis de lógica combinacional con diseño clásico

Semana 1 · Sistemas numéricos, códigos y compuertas lógicas

Ing. Luis Sinche Cueva · Período septiembre 2026 – marzo 2027

Objetivo de la clase

Al terminar la clase podrás



Convertir

números entre decimal, binario, octal y hexadecimal



Representar

números negativos en complemento a dos



Reconocer

los códigos BCD, Gray y ASCII y para qué sirve cada uno



Leer

las 7 compuertas lógicas y ubicarlas en un circuito integrado 74LS

Aporta a R1: diseña e implementa circuitos combinacionales básicos utilizando técnicas de diseño clásico.

Activación de conocimientos previos

Antes de empezar: tres preguntas

¿Qué hace un interruptor?

Recuerda Electricidad: ¿qué pasa con la corriente cuando está abierto y cuando está cerrado?

¿Cómo guarda tu celular la letra «A»?

Si la memoria solo guarda números… ¿qué número es la «A»?

¿Por qué 8, 16, 32 o 64 GB?

¿Por qué las memorias nunca vienen de 10, 20 o 50 GB?

Representación binaria

¿Por qué el computador usa solo 0 y 1?



0

Interruptor abierto: no pasa corriente, voltaje bajo

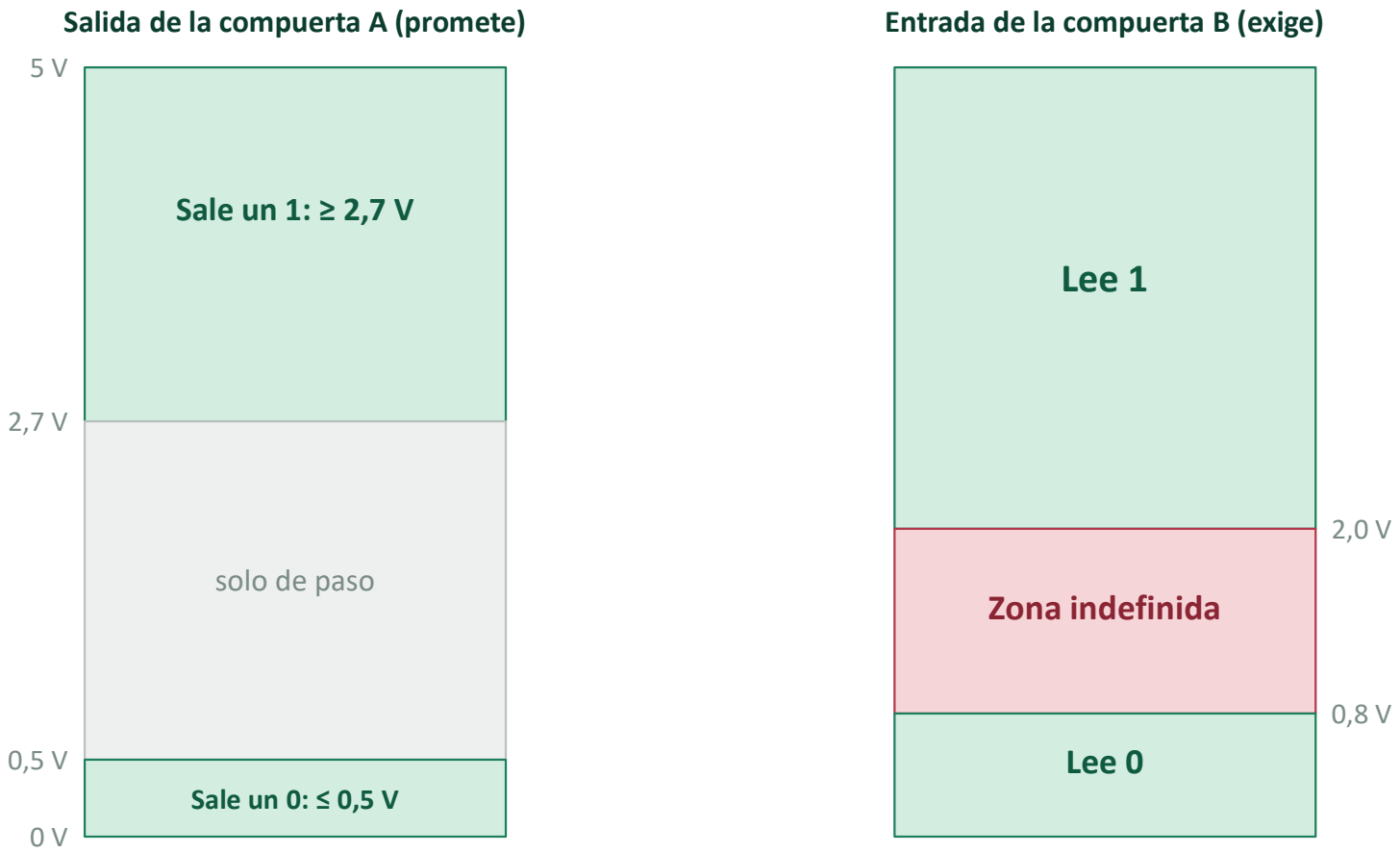


1

Interruptor cerrado: pasa corriente, voltaje alto

Un transistor es un interruptor controlado por voltaje.
Con solo dos estados es fácil distinguir uno del otro, aunque la señal llegue con ruido.

Un «1» no es un voltaje exacto: es un rango



Familia 74LS
Entre 0,8 y 2,0 V el chip no garantiza nada: puede leer 0 o 1.

Causa típica: una entrada sin conectar.

Margen de ruido: el colchón de seguridad

0,7 V

Margen alto = 2,7 – 2,0
Un 1 puede perder hasta 0,7 V en el camino y se sigue leyendo como 1.

0,3 V

Margen bajo = 0,8 – 0,5
Un 0 puede subir hasta 0,3 V por ruido y se sigue leyendo como 0.



Ejemplo de falla: sale un 0 de 0,5 V, un motor cercano induce 0,4 V de ruido → llegan 0,9 V → zona indefinida → error.

Sistema posicional: la posición da el valor

5	5	5
× 100	× 10	× 1
500	50	5

500 + 50 + 5 = 555

Contraejemplo: números romanos
En XV la V vale 5 esté donde esté.

Por eso sumar con romanos es tan incómodo.

En todo sistema posicional cada posición vale una potencia de la base.

Binario: las posiciones valen 1, 2, 4, 8…

8	4	2	1
1	0	1	1

$1011_2 = 8 + 0 + 2 + 1 = 11$

Bit = un dígito binario (binary digit)

Byte = 8 bits

Memoriza las potencias de 2

128	64	32	16	8	4	2	1

De decimal a binario: dividir entre 2

División	Cociente	Resto
25 ÷ 2	12	**1**
12 ÷ 2	6	**0**
6 ÷ 2	3	**0**
3 ÷ 2	1	**1**
1 ÷ 2	0	**1**



Leer los restos de abajo hacia arriba

$$25 = 11001_2$$

Comprobación: 16 + 8 + 1 = 25 ✓

Hexadecimal y octal: abreviaturas del binario

Hexadecimal: grupos de 4 bits desde la derecha

1001	1101
9	**D**

$157 = 10011101_2 = 9D_{16}$

Octal: grupos de 3 bits desde la derecha

010	011	101
2	**3**	**5**

$157 = 235_8$

Dec	Hex
10	**A**
11	**B**
12	**C**
13	**D**
14	**E**
15	**F**

Si falta, se completa con ceros a la izquierda

Actividad 1 · en parejas · 7 minutos

Reto de conversión

1. Convierte 45 a binario y a hexadecimal
2. Convierte $A7_{16}$ a decimal
3. Convierte 11100101_2 a hexadecimal y a octal

Reglas
Uno resuelve, el otro comprueba con los pesos. Luego cambian.

Al terminar, una pareja pasa a la pizarra por ejercicio.

Suma binaria: igual que en decimal, con acarreo

0 + 0	= 0
0 + 1	= 1
1 + 1	= 10
1 + 1 + 1	= 11

«10» = escribo 0 y llevo 1

```
acarreo   1 1 1
            1 0 1 1    (11)
          + 0 1 1 0    ( 6)
          ---------
          1 0 0 0 1    (17)
```

¿Y los números negativos? El circuito no tiene «–»



Un cuentakilómetros que retrocede desde 000 muestra 999

En binario de 4 bits pasa lo mismo: `0000 − 1 = 1111.` Acordamos que 1111 significa −1, 1110 significa −2… Ese acuerdo se llama complemento a dos.

Complemento a dos con 4 bits

Positivos: el bit izquierdo es 0

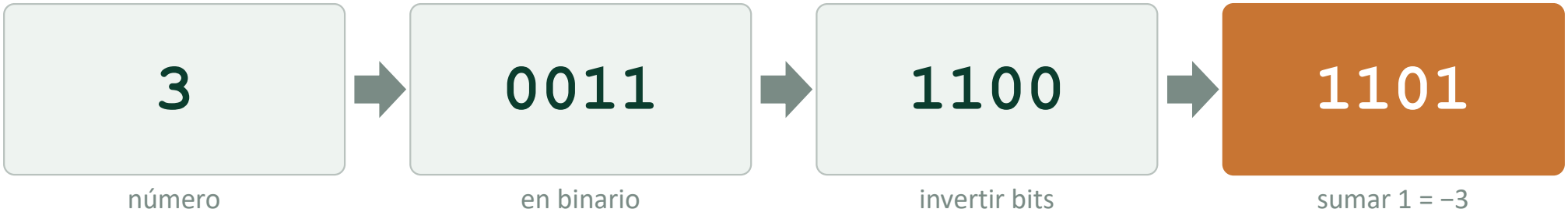
0000	0001	0010	0011	0100	0101	0110	0111
0	1	2	3	4	5	6	7

Negativos: el bit izquierdo es 1

1000	1001	1010	1011	1100	1101	1110	1111
−8	−7	−6	−5	−4	−3	−2	−1

Rango con 4 bits: de −8 a +7 · con 8 bits: de −128 a +127

Cómo obtener el negativo: invertir y sumar 1




```
5 − 3 = 5 + (−3)
  0101
+ 1101
------
1 0010   → 0010 = 2 ✓
```

La gran ventaja

Para restar no se necesita un circuito restador: el mismo sumador + inversores.

El bit que sobra a la izquierda se descarta.

Cuidado: el resultado debe caber en los bits

Con 4 bits: 7 + 1

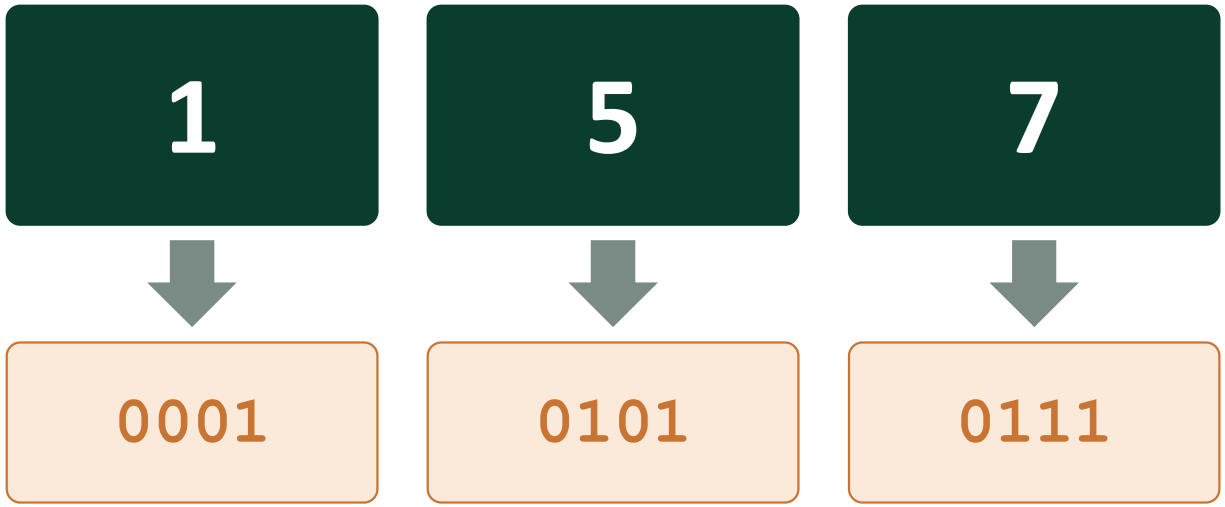
```
  0111
+ 0001
= 1000 → −8  ✗
```

```
// int en Java: 32 bits, comp. a dos
int x = 2147483647;
x = x + 1;
// x vale -2147483648
```



Desbordamiento: dos positivos dan un negativo (o al revés). El resultado no cabe en los bits disponibles. Lo detectaremos con hardware en la semana 4.

BCD: cada dígito decimal en 4 bits



Compara: 157 en binario natural es `10011101` (otra cosa).

¿Para qué?
Mostrar números en displays: relojes, calculadoras, contadores.

Las combinaciones 1010 a 1111 no se usan.

Código Gray: entre vecinos cambia un solo bit

1 bit	2 bits	3 bits
0	00	000
1	01	001
	11	011
	10	010
		110
		111
		101
		100

Binario: 3 → 4

`011 → 100`

¡cambian 3 bits a la vez! Un sensor puede leer valores falsos en el instante del cambio.

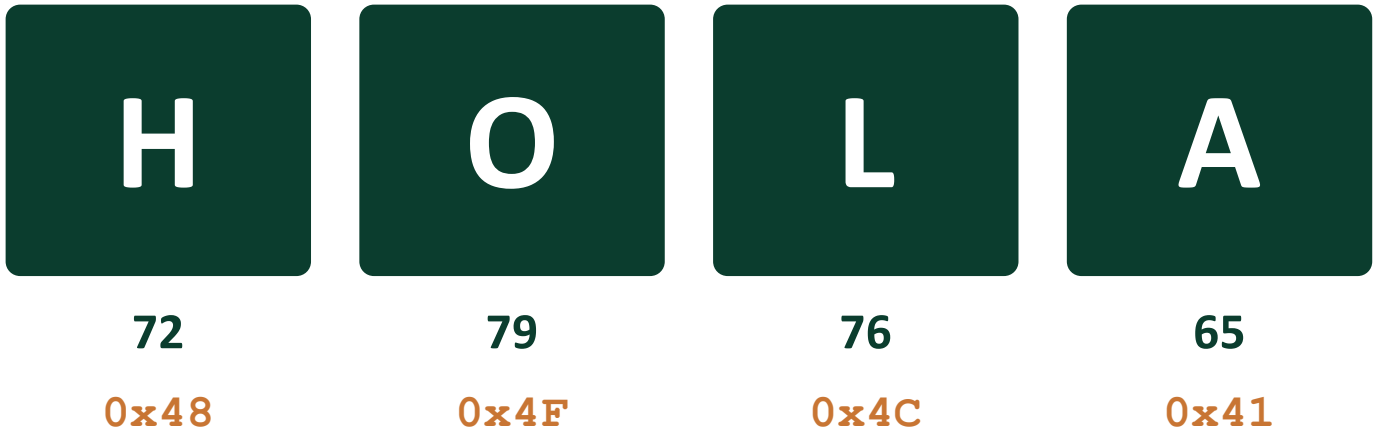
¿Dónde se usa?

Perillas y encoders rotativos.

El orden 00, 01, 11, 10 del mapa de Karnaugh (semana 3).

Método del espejo: copiar la lista al revés, anteponer 0 arriba y 1 abajo

ASCII: una tabla acordada de letras a números



Respuesta a la activación: tu celular guarda la «A» como el número 65 = 01000001.

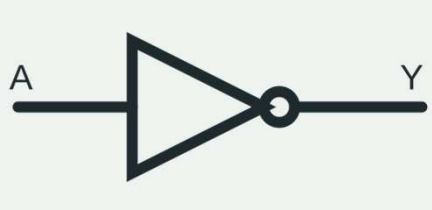
7 bits = 128 caracteres

Letras, dígitos, signos y controles.

La ñ y las tildes no están: para eso existe UTF-8.

Las compuertas básicas: NOT, AND, OR

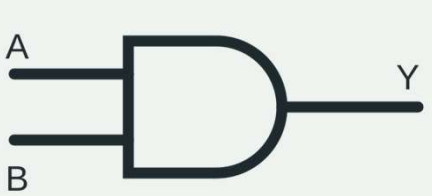
NOT



Invierte: Y = A'

A	Y
0	**1**
1	0

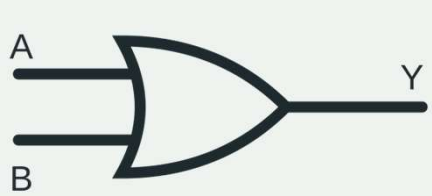
AND



1 solo si TODAS valen 1: Y = A·B

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	**1**

OR

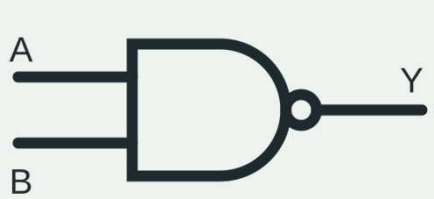


1 si AL MENOS UNA vale 1: Y = A+B

A	B	Y
0	0	0
0	1	**1**
1	0	**1**
1	1	**1**

Las negadas: NAND y NOR

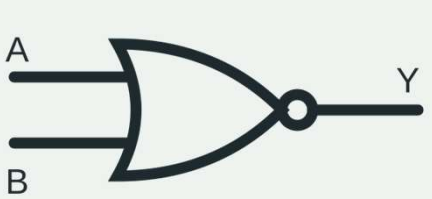
NAND



NOT-AND: 0 solo si todas valen 1

A	B	Y
0	0	**1**
0	1	**1**
1	0	**1**
1	1	0

NOR



NOT-OR: 1 solo si ninguna vale 1

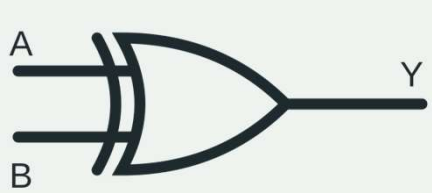
A	B	Y
0	0	**1**
0	1	0
1	0	0
1	1	0

El círculo en la salida significa «negado»: invierte el resultado.

Dato para la semana 3: con solo NAND (o solo NOR) se puede construir cualquier circuito.

Las de comparación: XOR y XNOR

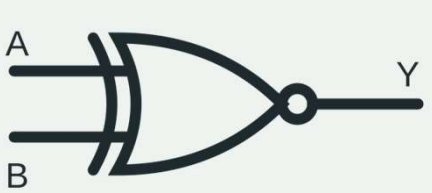
XOR



1 si las entradas son DISTINTAS

A	B	Y
0	0	0
0	1	**1**
1	0	**1**
1	1	0

XNOR



1 si las entradas son IGUALES

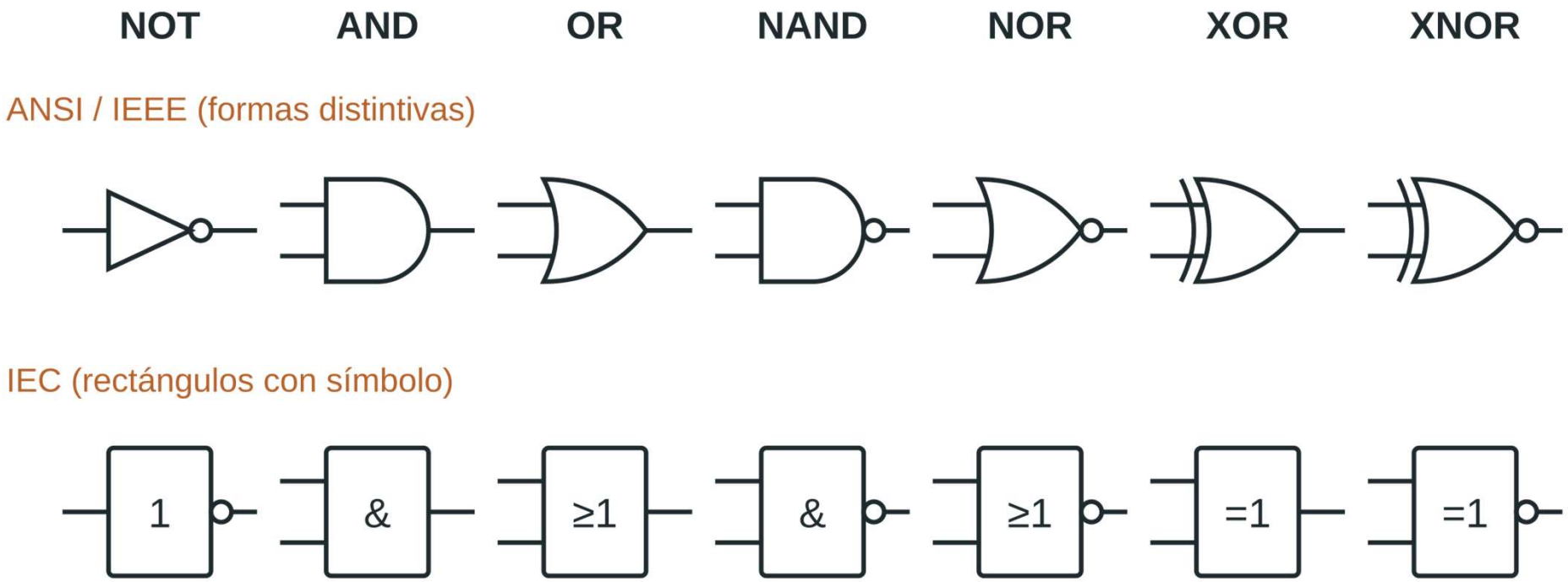
A	B	Y
0	0	**1**
0	1	0
1	0	0
1	1	**1**

Ejemplo cotidiano

La luz de una escalera con dos interruptores (abajo y arriba): cambia cada vez que mueves cualquiera de los dos.

Eso es una XOR.

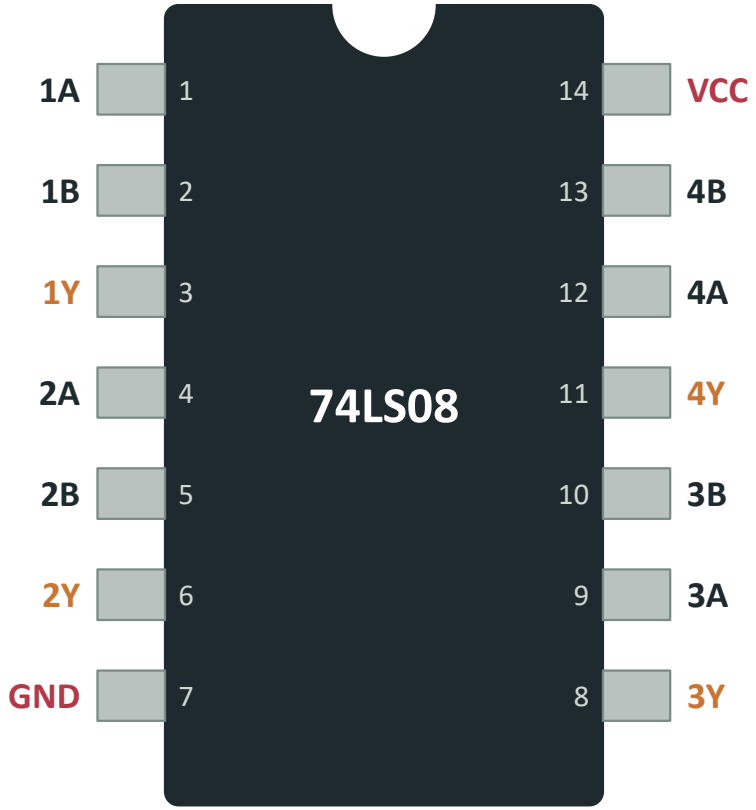
Dos normas para dibujar la misma compuerta



En el curso usaremos ANSI. Si el simulador muestra rectángulos, se cambia en sus opciones.

Demostración

Las compuertas vienen en circuitos integrados: 74LS08



Encapsulado DIP-14
dos filas de 7 pines; encaja en el protoboard

Contar los pines
muesca arriba; pin 1 arriba a la izquierda; sentido antihorario

Alimentación
pin 14 = +5 V · pin 7 = tierra

4 compuertas AND en un chip · A, B = entradas · Y = salida

Cómo leer el código impreso y buscar la hoja de datos

SN	74	LS	08	N
Fabricante	**Rango de uso**	**Familia / tecnología**	**Función**	**Encapsulado**
Texas Instruments	comercial (54 = militar)	LS = TTL; HC = CMOS	4 AND de 2 entradas	DIP de plástico



Buscar: «74LS08 datasheet». Preferir la web del fabricante (ti.com, onsemi.com).



¿Te venden 74HC08? Misma función y mismos pines; es la versión CMOS. Sirve igual.

Actividad 2 · taller dirigido · individual · 12 minutos

Del enunciado a la compuerta

1 · ¿Qué compuerta describe cada frase?

- a) La alarma suena solo si las dos puertas están abiertas.
- b) La luz del pasillo se enciende si se pulsa cualquiera de los dos botones.
- c) El juego marca «diferentes» si los dos jugadores eligen distinto.
- d) La bomba de agua se apaga cuando el sensor de nivel está activo.

2 · Tabla de verdad de

$$Y = A \cdot B + C'$$

- 3 entradas → ¿cuántas filas?
- Agrega columnas intermedias: A·B y C'
- Luego la columna final Y
- Bonus: convierte cada fila ABC a decimal

Resumen

Lo que nos llevamos hoy

Dos estados

0 y 1 son rangos de voltaje; entre ellos hay una zona indefinida

Bases

binario = potencias de 2; hex = 4 bits; octal = 3 bits

Complemento a dos

negativo = invertir + 1; restar = sumar el negativo

Códigos

BCD para displays, Gray para sensores, ASCII para texto

7 compuertas

NOT, AND, OR, NAND, NOR, XOR, XNOR y su tabla de verdad

Hoja de datos

pines, alimentación (14 y 7) y función del 74LS08

Para la próxima sesión

Lectura

Apartado de sistemas numéricos y códigos del texto base (aula virtual).

Cuestionario 01

Resolverlo en el aula virtual después de la lectura.

Instalar Logisim-evolution

Desde GitHub (logisim-evolution → Releases): instalador .msi para Windows o .dmg para macOS.

Martes · práctica APE 01

Comprobación de compuertas en el simulador. Trae tu laptop si puedes.